অধ্যায় ২১

JavaScript Design Patterns, Software Architecture এবং Scalable Application Structure

Hands-on Project: TravelFlow Booking Manager

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Design Pattern ও Architecture-এর পার্থক্য বুঝবে, (২) Factory, Strategy, Observer, Repository এবং Dependency Injection ব্যবহার করতে পারবে, (৩) Domain, Application ও Presentation Layer আলাদা করতে পারবে, (৪) State Transition এবং Error Handling সঠিকভাবে ডিজাইন করতে পারবে, (৫) Testable Booking Application তৈরি করতে পারবে।

২১.১ ছোট কোড থেকে বড় Application: সমস্যাটা কোথায়?

একটি ৫০ লাইনের JavaScript ফাইলে DOM Update, Price Calculation ও Storage একসঙ্গে লিখলে প্রথমে সহজ মনে হয়। কিন্তু Project বড় হলে একটি পরিবর্তনে একাধিক Feature ভেঙে যায়। উদাহরণস্বরূপ, Booking Price যদি Button Click Handler-এর মধ্যে হিসাব করা হয়, সেই হিসাব Unit Test, Server API বা অন্য UI-তে পুনর্ব্যবহার কঠিন।

```javascript
// Anti-pattern: UI এবং Business Logic এক জায়গায়।
document.querySelector('#save').addEventListener('click', () => {
  const price = Number(document.querySelector('#price').value);
  localStorage.setItem('booking', JSON.stringify({ price }));
});
```

এখানে User নিজেই Price পরিবর্তন করতে পারে। Storage Failure, Validation, State Transition ও Future API Migration-এর ব্যবস্থা নেই। **Separation of Concerns** অনুসারে দায়িত্বগুলো আলাদা করা ভালো।

২১.২ Design Pattern বনাম Architecture

Design Pattern হলো পুনরাবৃত্ত Design Problem সমাধানের পরীক্ষিত কাঠামো; যেমন Observer বা Factory। **Architecture** হলো পুরো System-এর Component, Dependency ও Data Flow সাজানোর সিদ্ধান্ত; যেমন Layered বা Hexagonal Architecture। Pattern কোনো বাধ্যতামূলক Recipe নয়—প্রয়োজন ছাড়া Pattern ব্যবহার করলে অকারণ Complexity বাড়ে।

Concept	Focus	Example
Factory	Object creation	`createBookingService()`

Concept	Focus	Example
Strategy	Swappable algorithm	Discount strategies
Observer	Event notification	Booking created event
Repository	Data access abstraction	Memory/Browser storage
Dependency Injection	Dependency externalize	Service receives repository
Layered Architecture	Responsibility separation	UI → Service → Domain → Repository

২১.৩ Single Responsibility এবং Cohesion

Single Responsibility Principle (SRP) বোঝায়: একটি Module-এর পরিবর্তনের প্রধান কারণ একটি হওয়া উচিত। UI Module User Interaction সামলাবে, Domain Module Business Rules, Repository Data Persistence। Cohesion বেশি এবং Coupling কম থাকলে Refactoring ও Testing সহজ হয়।

```javascript
// Domain: কোনো Browser API নেই।
export function calculateTotal(unitPrice, travelers) {
  if (!Number.isFinite(unitPrice) || unitPrice < 0) throw new Error('Invalid price');
  if (!Number.isSafeInteger(travelers) || travelers < 1) throw new Error('Invalid travelers');
  return unitPrice * travelers;
}
```

এটি Browser, Node.js এবং Test Runner—সব জায়গায় চলতে পারে।

২১.৪ Factory Pattern

Factory Object Creation-এর Details লুকায়। Constructor Call ছড়িয়ে না রেখে একটি Function দিয়ে Configured Instance তৈরি করা যায়।

```javascript
function createNotifier(channel) {
  if (channel === 'console') return { send: message => console.log(message) };
  if (channel === 'silent') return { send: () => {} };
  throw new Error('Unknown channel');
}
const notifier = createNotifier('console');
notifier.send('Booking created');
```

Factory Test-এ Fake Dependency দেওয়ার পথও খুলে দেয়।

২১.৫ Strategy Pattern

একাধিক Algorithm একই Interface অনুসরণ করলে Strategy উপকারী। যেমন বিভিন্ন Discount Rule:

```javascript
const strategies = {
  regular: total => total,
  member: total => total * 0.95,
  seasonal: total => total * 0.90
};
function applyDiscount(total, type = 'regular') {
  const strategy = strategies[type];
  if (!strategy) throw new Error('Unknown discount strategy');
  return Math.round(strategy(total));
}
console.log(applyDiscount(10000, 'member')); // 9500
```

Real Payment বা Price-এর ক্ষেত্রে Discount Server-side পুনরায় যাচাই করতে হবে।

২১.৬ Observer / Publish–Subscribe Pattern

একটি Booking তৈরি হলে UI Refresh, Analytics ও Notification—একাধিক Consumer-কে জানানোর প্রয়োজন হতে পারে। Observer Pattern-এ Producer সরাসরি Consumer-এর Implementation জানে না।

```javascript
function createEventBus() {
  const handlers = new Map();
  return {
    on(type, handler) {
      const group = handlers.get(type) ?? new Set();
      group.add(handler);
      handlers.set(type, group);
      return () => group.delete(handler);
    },
    emit(type, payload) {
      for (const handler of handlers.get(type) ?? []) handler(payload);
    }
  };
}
```

সতর্কতা: Synchronous `emit()`-এ Listener Exception ছুড়লে পরের Listener আর নাও চলতে পারে। Production System-এ Error Isolation, Logging এবং Async Delivery Semantics আলাদাভাবে ডিজাইন করতে হয়। Unsubscribe না করলে Memory Leak হতে পারে।

২১.৭ Repository Pattern

Business Logic যেন সরাসরি `localStorage` বা Database Query না জানে—Repository সেই Boundary তৈরি করে।

```javascript
function createMemoryRepository() {
  const records = new Map();
  return {
    async get(id) { return records.get(id) ?? null; },
    async save(record) { records.set(record.id, record); },
    async list() { return [...records.values()]; }
  };
}
```

`async` Interface ব্যবহার করায় পরবর্তীতে HTTP API বা Database Repository-তে বদলানো সহজ। তবে Production Repository-তে Concurrency, Transactions, Error Handling ও Authorization প্রয়োজন।

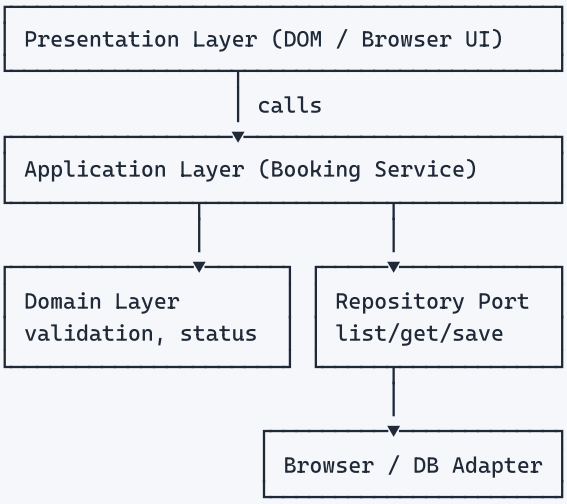
২১.৮ Dependency Injection (DI)

Service-এর ভেতরে Storage ও Event Bus তৈরি না করে বাইরে থেকে পাঠালে Testability বাড়ে।

```javascript
function createService({ repository, events }) {
  return {
    async save(record) {
      await repository.save(record);
      events.emit('saved', record);
      return record;
    }
  };
}
```

এই পদ্ধতিকে **Constructor/Function Injection** বলা যায়। এটি Framework ছাড়াই করা যায়।

২১.৯ Layered Architecture ও Dependency Direction



Domain Layer Browser DOM-এর উপর নির্ভর করবে না। Repository-এর Storage Implementation বদলালেও Service-এর Public Contract একই থাকতে পারে।

২১.১০ State Machine ও Business Rules

Booking Status যেকোনো Value থেকে যেকোনো Value-তে পরিবর্তন করা উচিত নয়। এই Project-এ Transition Rules:

Current	Allowed Next
`pending`	`confirmed`, `cancelled`
`confirmed`	`cancelled`
`cancelled`	কোনো Transition নয়

```javascript
const allowed = {
  pending: ['confirmed', 'cancelled'],
  confirmed: ['cancelled'],
  cancelled: []
};
function canTransition(current, next) {
  return (allowed[current] ?? []).includes(next);
}
```

State Machine Invalid Transition আটকায় এবং Workflow-কে স্পষ্ট করে।

২১.১১ Practical Project — TravelFlow Booking Manager

Project Goal: একটি Educational Booking Dashboard তৈরি করা, যেখানে Customer Package নির্বাচন করে Booking Request তৈরি করবে এবং Status Update করা যাবে। Architecture-এ Domain, Repository, Event Bus, Service ও UI আলাদা থাকবে। এটি বাস্তব Booking Engine নয়।

Folder Structure

```
source-code/chapter-21/
├── index.html
├── style.css
├── booking-core.mjs
├── repository.mjs
├── events.mjs
├── service.mjs
├── app.mjs
└── booking-core.test.mjs
```

গুরুত্বপূর্ণ File Responsibilities

`booking-core.mjs`: `validateRequest()`, `createBooking()`, `transitionBooking()`, `summarize()`। User Input থেকে Price গ্রহণ না করে Catalog থেকে Price হিসাব করা হয়।

`repository.mjs`: `createMemoryRepository()` এবং `createBrowserRepository()`। দুটিই `list/get/save` Interface অনুসরণ করে।

`events.mjs`: `createEventBus()` এবং Unsubscribe Function।

`service.mjs`: `createBookingService()`—Booking Creation ও State Change-এর Orchestration।

`app.mjs`: DOM Rendering, Form Submission, Event Listener ও UI Feedback। User-provided Text প্রদর্শনে `textContent` ব্যবহার করা হয়েছে।

সম্পূর্ণ Runnable Source Code ও Tests Companion Source Code-এর `source-code/chapter-21/` Folder-এ রয়েছে। অধ্যায়ের এই ব্যাখ্যার সঙ্গে ওই Files পড়বে।

Run Instructions

```
TERMINAL
# Project folder থেকে:
python -m http.server 8000
# Browser: http://localhost:8000

# Node.js installed থাকলে:
node --test booking-core.test.mjs
```

ES Modules Browser-এ সাধারণত `file://` দিয়ে চালানোর বদলে Local HTTP Server ব্যবহার করবে।

Data Flow

1. User Form Submit করে।
2. UI → Service-এ Request পাঠায়।
3. Service → Domain Validation ও Booking Creation করে।
4. Service → Repository-তে Save করে।
5. Service → Event Bus-এ `booking:created` Emit করে।
6. UI Listener → নতুন Data Load করে Table Refresh করে।

Important Limitations

এই Browser Demo-র `localStorage` Trusted Database নয়। User Data পরিবর্তন করতে পারে। Multi-user Access, Authentication, Authorization, Real Booking Confirmation, Inventory Locking, Payment, Audit Trail এবং Server-side Validation এখানে নেই। Production Implementation-এ Backend API ও Database Transaction প্রয়োজন।

২১.১২ Testing Strategy

এই Project-এ Unit ও Integration Tests দুটোই আছে। Unit Test-এ Price Calculation, Invalid Input ও State Transition যাচাই করা হয়। Integration Test-এ Service, Repository এবং Event Bus একসঙ্গে কাজ করছে কি না দেখা হয়।

```javascript
import test from 'node:test';
import assert from 'node:assert/strict';
import { validateRequest } from './booking-core.mjs';

test('price comes from catalog', () => {
  const result = validateRequest({
    customer: 'Amina Khan', packageId: 'cox', travelers: 2, total: 1
  });
  assert.equal(result.total, 13000);
});
```

Browser UI-এর Keyboard Navigation, Accessibility, LocalStorage Error Handling এবং Manual End-to-End Testing আলাদা Review-এর অংশ।

২১.১৩ Common Architecture Mistakes

- **God Object:** সব Business Rule ও UI Logic একটি Module-এ রাখা।
- **Overengineering:** ছোট Application-এ অপ্রয়োজনীয় ১০টি Pattern যোগ করা।
- **Hidden Dependency:** Service-এর মধ্যে সরাসরি Global `localStorage` ব্যবহার করা।
- **Mutable Shared State:** Reference Copy-এর কারণে অনিচ্ছাকৃত পরিবর্তন।
- **Missing Error Boundary:** Storage Failure-এর পরে Success Message দেখানো।
- **Trusting the Client:** Browser-এর Price/Status-কে Authoritative ধরে নেওয়া।
- **Unmanaged Listeners:** Event Subscription পরিষ্কার না করা।

২১.১৪ অনুশীলন — ১০টি

১. SRP অনুযায়ী একটি Monolithic Form Handler-কে তিনটি Function-এ ভাগ করো।

২. `createNotifier('console')` ও `createNotifier('silent')` Factory তৈরি করো।

৩. Student Discount-এর জন্য নতুন Strategy যোগ করো।

৪. Event Bus-এর `on()` Function থেকে Unsubscribe Return করো।

৫. Repository-তে `remove(id)` Method যোগ করো।

৬. `confirmed → pending` Transition কেন Invalid, ব্যাখ্যা করো।

৭. Booking Service-এ `repository` Mock করে Unit Test লেখো।

৮. `localStorage`-এ Invalid JSON থাকলে App কী করবে, পরিকল্পনা করো।

৯. Domain Layer থেকে DOM Dependency সরানোর উদাহরণ দাও।

১০. TravelFlow-এ `booking:updated` Event দিয়ে Audit Logger যুক্ত করো।

Sample Solutions

১. `readForm()` Input পড়ে; `validateRequest()` Business Rule যাচাই করে; `renderResult()` UI Update করে।

২.

```javascript
const createNotifier = type => type === 'console'
  ? { send: message => console.log(message) }
  : { send: () => {} };
```

৩.

```javascript
strategies.student = total => total * 0.92;
```

৪.

```javascript
const off = bus.on('saved', console.log);
off(); // Listener removed
```

৫.

```javascript
async remove(id) { return records.delete(id); }
```

৬. Confirmed Booking-কে Pending করলে Workflow History অসংগত হতে পারে। নতুন Business Process ছাড়া Backward Transition নিষিদ্ধ রাখা সহজ ও নিরাপদ।

৭.

```javascript
const saved = [];
const repository = { save: async x => saved.push(x) };
```

৮. JSON Parse `try/catch`-এ রেখে Safe Fallback দেখাতে হবে; Corrupt Data-কে চুপচাপ মুছে না দিয়ে Recovery/Export Strategy রাখা ভালো।

৯. `document.querySelector()`-এর বদলে Domain Function-এ Plain Object Input দাও।

১০.

```javascript
const unsubscribe = events.on('booking:updated', booking => {
  console.info('Updated booking', booking.id, booking.status);
});
```

২১.১৫ MCQ — ১০টি

১. Repository Pattern মূলত কী Abstract করে? A) CSS B) Data Access C) Typography D) HTML

২. Observer Pattern-এর উদ্দেশ্য কী? A) Object creation B) Event notification C) Database migration D) Password hashing

৩. Dependency Injection-এর সুবিধা কী? A) Testing easier B) No errors ever C) Browser faster always D) No security needed

৪. Domain Layer-এ সাধারণত কোনটি থাকা উচিত? A) DOM Event Listener B) Business Rules C) CSS Grid D) Browser History

৫. Factory Pattern কী করে? A) Creates configured objects B) Compresses images C) Encrypts tokens D) Renders tables

৬. `cancelled → confirmed` এই Project-এ কী? A) Valid B) Invalid C) Always automatic D) CSS state

৭. `textContent` ব্যবহারের একটি কারণ কী? A) Safe plain-text rendering B) Faster payment C) Password hashing D) DB indexing

৮. Browser `localStorage` কি Trusted Server Database? A) Yes B) No C) Only Chrome D) Only HTTPS

৯. Strategy Pattern কখন উপযোগী? A) Swappable algorithms B) One fixed color C) One paragraph D) Static HTML only

১০. Service Layer সাধারণত কী করে? A) Orchestrates use cases B) Sets font family C) Manages DNS D) Paints pixels

MCQ Answers ও ব্যাখ্যা

1. **B** — Data Persistence-এর Implementation আড়াল করে।
2. **B** — Event ঘটলে Subscriber-দের জানায়।
3. **A** — Fake/Mock Dependency দিয়ে Test সহজ হয়।
4. **B** — Business Validation ও State Rule Domain-এর দায়িত্ব।
5. **A** — Object Creation-এর Logic Encapsulate করে।
6. **B** — State Transition Rules অনুমতি দেয় না।
7. **A** — Plain Text HTML হিসেবে Parse হয় না।
8. **B** — Browser Storage User পরিবর্তন করতে পারে।
9. **A** — Algorithm পরিবর্তনযোগ্য হলে Strategy উপকারী।
10. **A** — Use Case-এ বিভিন্ন Layer সমন্বয় করে।

২১.১৬ Chapter Recap ও পরবর্তী অধ্যায়

Design Patterns কোনো Magic Solution নয়; সঠিক Responsibility, Testable Boundaries এবং Explicit Business Rules-এর সঙ্গে ব্যবহার করলেই উপকার পাওয়া যায়। TravelFlow Project-এ Factory, Observer, Repository, Dependency Injection এবং Layered Architecture একসঙ্গে প্রয়োগ করেছি। পরবর্তী অধ্যায় **JavaScript Testing Mastery**-তে Unit, Integration, Mocking, TDD ও End-to-End Testing আরও গভীরভাবে শিখব।